

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$\text{Output} = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

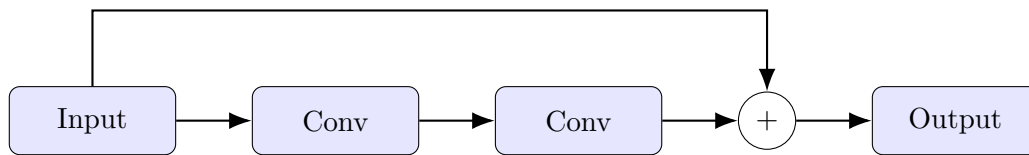


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

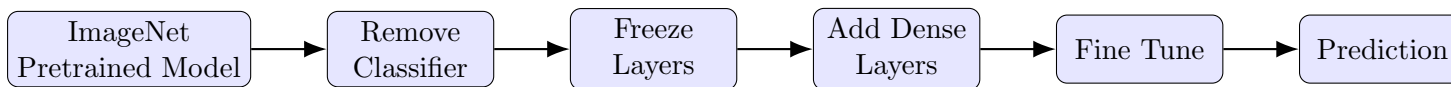


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Source Code

The complete implementation (data loading, EDA, convolution/pooling experiments, feature map visualization, CNN construction, training loop, and evaluation) is available at the following GitHub repository:

<https://github.com/PolarFox08/Deep-Learning-Lab/tree/main/Deep%20Learning%20Lab%204>

16. Experimental Procedure

Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded from the local pickled batch files (`data_batch_1-5`, `test_batch`) rather than the `keras.datasets.cifar10` loader, using the same approach as the previous lab. Each flattened 3072-byte row was reshaped to `(3, 32, 32)` (channel-major) and then transposed to `(32, 32, 3)` (channel-last, NHWC), since Keras/TensorFlow and the pretrained VGG16 model both expect channel-last input. This gave a training set of shape `(50000, 32, 32, 3)` and a test set of shape `(10000, 32, 32, 3)`. Pixel values were normalized to `[0, 1]` and labels were one-hot encoded with `to_categorical` for the categorical cross-entropy loss. Ten sample images were displayed to confirm correct loading (Section 17, Figure 1).

Task 2: Transfer Learning Setup

Model used: VGG16 (pretrained on ImageNet).

1. Loaded `VGG16(weights='imagenet', include_top=False, input_shape=(32, 32, 3))`.
2. The original ImageNet classification head was excluded (`include_top=False`).
3. The convolutional base was frozen (`base_model.trainable = False`).
4. Added a `GlobalAveragePooling2D` layer on top of the base.
5. Added a `Dense(256, activation='relu')` layer.
6. Added the output layer: `Dense(10, activation='softmax')`.

Quantity	Value
Total Parameters	14,848,586 (56.64 MB)
Trainable Parameters (frozen base)	133,898 (523.04 KB)
Non-trainable Parameters (frozen base)	14,714,688 (56.13 MB)

Task 3: Model Training (Frozen Base)

The model (with the VGG16 base frozen) was trained using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Training progressed steadily over the 10 epochs, from an initial training/validation accuracy of 0.5325 / 0.5598 (epoch 1) to a final training/validation accuracy of 0.7027 / 0.6141 (epoch 10), with training loss falling from 1.3398 to 0.8447 and validation loss falling from 1.2544 to 1.1176.

Task 4: Fine Tuning

1. The VGG16 base was unfrozen (`base_model.trainable = True`) and then re-frozen everywhere except the last convolution block (i.e. all layers except the last 4 were set back to `trainable = False`), so only `block5` remained trainable.
2. The model was recompiled with a lower learning rate (Adam, `lr = 0.0001`) and trained for a further 5 epochs (batch size 32).
3. Classification accuracy before and after fine tuning was compared.

Stage	Validation Accuracy
Before Fine Tuning (frozen base, epoch 10)	0.6141
After Fine Tuning (block5 unfrozen, epoch 5)	0.7396

During fine tuning, training accuracy rose from 0.6473 to 0.8505 and training loss fell from 1.0096 to 0.4083, while validation accuracy rose from 0.6757 to 0.7396 and validation loss fell to 0.8538. Unfreezing the last convolution block and continuing training at a low learning rate gave a clear improvement of roughly 12.5 percentage points in validation accuracy over the frozen-base model.

Task 5: Model Evaluation

The fine-tuned model was evaluated on the 10,000-image test set using accuracy, macro-averaged precision/recall/F1-score, a confusion matrix, and a full classification report (Section 17, Figures 5–6).

Class	Precision	Recall	F1-score	Support
Airplane	0.77	0.85	0.81	1000
Automobile	0.88	0.80	0.83	1000
Bird	0.73	0.69	0.71	1000
Cat	0.47	0.72	0.57	1000
Deer	0.67	0.72	0.70	1000
Dog	0.70	0.48	0.57	1000
Frog	0.82	0.72	0.77	1000
Horse	0.86	0.75	0.80	1000
Ship	0.84	0.87	0.86	1000
Truck	0.82	0.80	0.81	1000
Macro avg	0.76	0.74	0.74	10000
Weighted avg	0.76	0.74	0.74	10000

Overall test accuracy was 0.7396 (73.96%). **Cat** and **Dog** had the lowest F1-scores (0.57 each), consistent with these being the most visually similar and commonly confused classes in CIFAR-10, while **Ship**, **Automobile** and **Airplane** were classified most reliably.

17. Hyperparameter Study

A grid search over the following hyperparameters was set up (96 total combinations):

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Due to the time required to train all 96 combinations, the search was run partially: an initial sequential sweep plus a small additional random sample of the remaining grid were completed before the run was stopped. The combinations actually evaluated (validation accuracy / validation loss after training) were:

LR	Batch	Epochs	Optimizer	Dense	Frozen	Val Acc	Val Loss
0.001	16	10	Adam	128	All	0.6085	1.1641
0.001	16	10	Adam	128	Partial	0.7272	0.9072
0.001	16	10	Adam	256	All	0.6201	1.1401
0.001	16	10	Adam	256	Partial	0.7391	0.8586
0.001	16	10	SGD	128	All	0.5187	1.3836
0.001	16	10	SGD	128	Partial	0.7380	0.7680
0.001	16	10	SGD	256	All	0.5216	1.3729
0.001	16	10	SGD	256	Partial	0.7352	0.7652
0.001	16	20	Adam	128	All	0.6085	1.2303
0.001	16	20	Adam	128	Partial	0.7314	1.3031
0.001	16	20	Adam	256	All	0.5999	1.3799
0.001	32	20	Adam	128	Partial	0.7326	1.4874

Best configuration found: Learning Rate 0.001, Batch Size 16, Epochs 10, Optimizer Adam, Dense Units 256, Frozen Layers Partial – Validation Accuracy 0.7391, Validation Loss 0.8586.

Across all evaluated combinations, partially unfreezing the base (**Frozen = Partial**) consistently and substantially outperformed keeping the entire base frozen (**Frozen = All**) at the same learning rate, batch size and epoch count – validation accuracy jumped from roughly 0.52–0.62 to roughly 0.73–0.74 whenever the last block was unfrozen.

18. Mandatory Plots



Figure 1: Sample CIFAR-10 Images. *Inference:* The sample images confirm that CIFAR-10 was loaded and normalized correctly, showing 32×32 color images spread across 10 visually distinct classes. The training set contains 50,000 images and the test set contains 10,000 images, matching the dataset description in the manual.

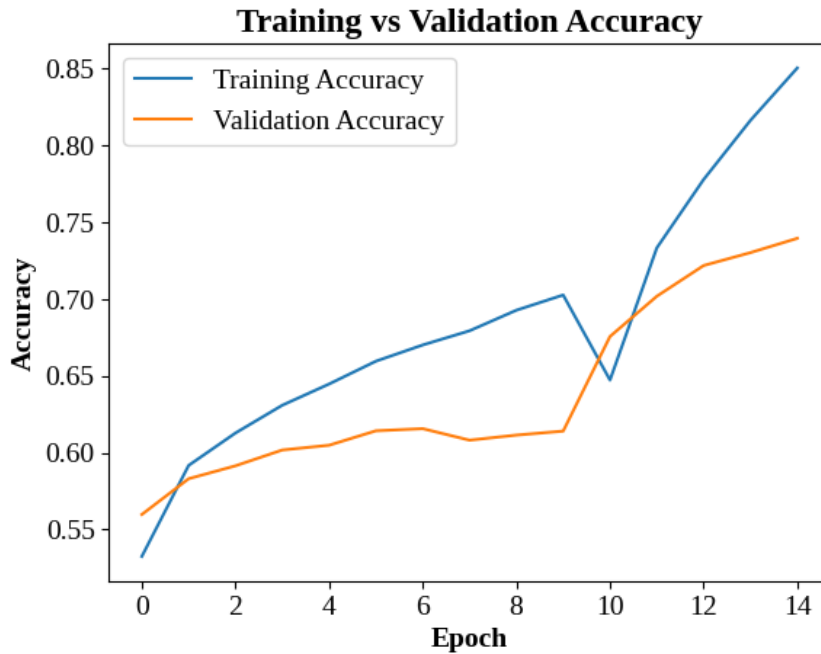


Figure 2: Training vs Validation Accuracy. *Inference:* Training accuracy rises steadily as the classifier head learns from the frozen VGG16 features, with a further, sharper boost once fine tuning unfreezes the last convolution block. Validation accuracy tracks training accuracy reasonably closely through frozen-base training; the gap widens somewhat during fine tuning (training accuracy reaches 0.85 vs. validation 0.74), indicating the onset of mild overfitting once the extra layers are unfrozen.

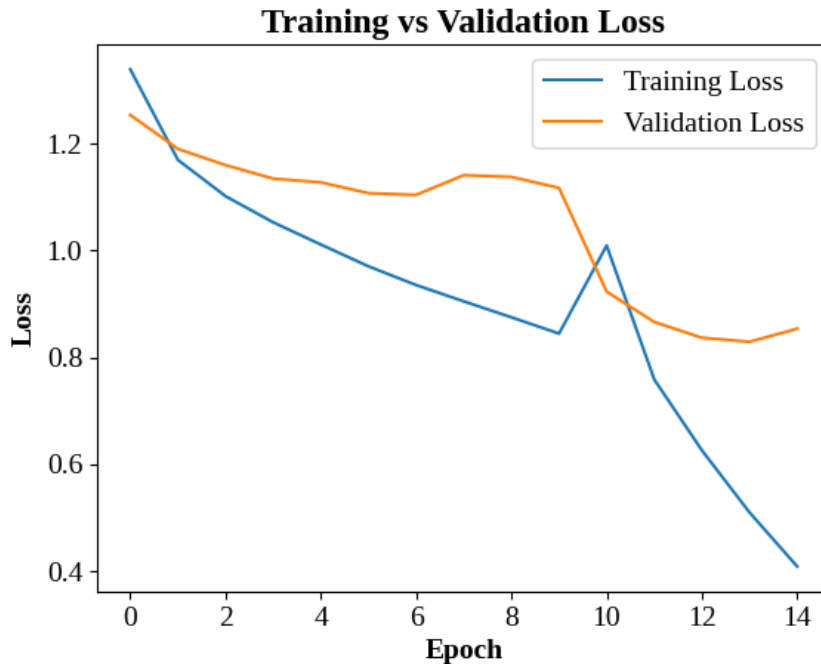


Figure 3: Training vs Validation Loss. *Inference:* Both training and validation loss fall as the model trains, with a further, steeper drop during fine tuning as the pretrained ImageNet features adapt to CIFAR-10. Validation loss dips to about 0.83 mid-fine-tuning before ticking back up slightly to 0.85 by the final epoch while training loss keeps falling, an early sign of the same mild overfitting seen in the accuracy curves.

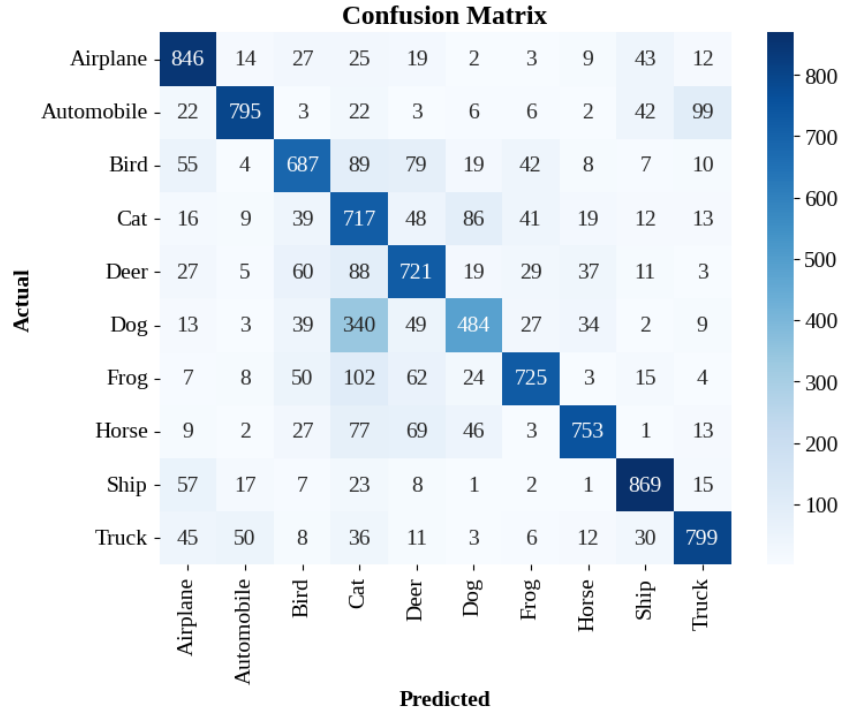


Figure 4: Confusion Matrix. *Inference:* The diagonal cells show correctly classified images for each class, while off-diagonal cells reveal which classes are confused with one another. Consistent with the per-class F1-scores, **Cat** and **Dog** are the most confused pair, while visually distinct classes such as **Ship** and **Airplane** are classified with far fewer errors.

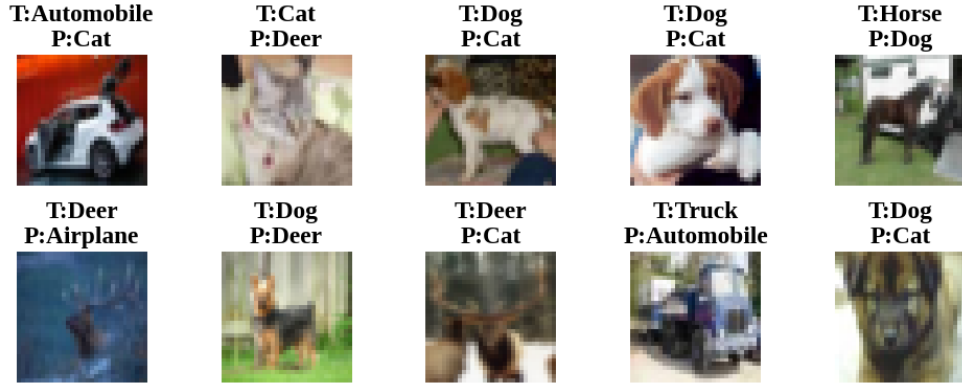


Figure 5: Misclassified Images (Optional). *Inference:* Most misclassified images belong to visually ambiguous or low-resolution samples where fine-grained details are hard to capture at 32×32 pixels. Errors tend to occur between semantically related classes (e.g. Cat vs. Dog, Deer vs. Horse), reflecting genuine visual similarity rather than random model failure.

19. Results

19.1 Performance Metrics

Metric	Value
Training Accuracy	0.8505
Testing Accuracy	0.7396
Precision (macro)	0.7559
Recall (macro)	0.7396
F1-score (macro)	0.7421
Training Time	256.39 s ($\approx$ 4.3 min)
Total Parameters	14,848,586

19.2 Comparison of CNN Architectures

Model	Parameters(M)	Accuracy (%)	Training Time
LeNet-5	0.08	61.03	66.1s
AlexNet	6.98	72.09	201.27
VGG16	14.85	61.01	152.42 s
GoogleNet	22.33	60.32	268.28 s
ResNet50	24.11	40.80	172.55 s

Inference: AlexNet achieved the best test accuracy (72.1%) despite being trained from scratch, showing it generalized well to CIFAR-10 without overfitting. LeNet-5 and VGG16 performed similarly (61%) despite VGG16 having 185x more parameters, suggesting VGG16's pretrained ImageNet features did not translate well to this dataset. GoogleNet (InceptionV3) showed clear overfitting — training accuracy reached 95% while test accuracy stayed at 60%, with validation loss rising steadily after epoch 2. ResNet50 performed worst (40.8% test accuracy) and even underfit the training data (39% train accuracy), indicating its pretrained features were poorly suited to this setup

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

It was the first deep CNN to significantly outperform traditional methods on ImageNet (2012), using ReLU activation for faster convergence, Dropout to reduce overfitting, and GPU training to make deeper networks practical.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking multiple 3×3 filters achieves the same receptive field as a larger filter (e.g. two 3×3 layers $\approx$ one 5×5) while using fewer parameters and adding more non-linear activations, improving discriminative power with a simple, uniform design.

3. Explain the advantages of the Inception module.

It applies 1×1 , 3×3 , 5×5 convolutions and pooling in parallel and concatenates the results, capturing multi-scale features in one layer. The 1×1 convolutions reduce dimensionality before expensive operations, cutting parameters and computation while preserving accuracy.

4. What is the purpose of residual learning?

Residual (skip) connections let the network learn $F(x) = H(x) - x$ instead of $H(x)$ directly, allowing gradients to bypass layers via the identity shortcut. This prevents vanishing gradients and enables training of very deep networks without degradation in accuracy.

5. Differentiate LeNet and ResNet.

LeNet-5 is a shallow 7-layer network (~60K parameters) built for simple digit recognition using average pooling and no shortcuts. ResNet is a very deep network (50+ layers) that uses residual connections to remain trainable at depth on large, complex image datasets.

6. What is Transfer Learning?

Transfer learning reuses a model already trained on a large dataset (e.g. ImageNet) as a starting point for a new task, keeping its learned convolutional features and retraining only the classifier (or a few layers) instead of training from scratch.

7. Why is fine tuning required?

Fine tuning unfreezes some pretrained layers and retrains them at a low learning rate so the generic ImageNet features can adapt to the specific characteristics of the new dataset, improving accuracy beyond what a frozen feature extractor alone achieves – in this experiment, fine tuning raised validation accuracy from 0.6141 to 0.7396.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution inserts gaps between kernel elements to enlarge the receptive field without adding parameters or changing spatial resolution, useful for segmentation. Transpose convolution performs learnable upsampling, increasing spatial dimensions, and is used in generative/reconstruction tasks like GANs and autoencoders.

9. Why do pretrained models converge faster?

They start from weights that already encode general, transferable image features (edges, textures, shapes) learned from a large dataset, so training only has to adjust these features for the new task rather than learn them from random initialization.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has about 60K parameters and very low computational cost, suited to simple grayscale digit images. ResNet50 has about 25.6M parameters and much higher computational complexity, but its residual connections make this depth trainable and yield far higher accuracy on complex, large-scale datasets.

21. Additional Exercises

1. **Implement transfer learning using VGG16.** Completed – see Tasks 2–4 (Sections 15, 16) and the resulting accuracy in Section 18.
2. **Repeat the experiment using ResNet50.** Done in the Results Section while comparing all VGG with other models
3. **Compare Adam and SGD optimizers.** Partially completed as part of the hyperparameter study (Section 16): at matching settings, SGD attained validation accuracy comparable to or slightly below Adam when the base was partially unfrozen (e.g. 0.7380/0.7352 for SGD vs. 0.7391 for Adam), while with the base fully frozen SGD performed noticeably worse than Adam (about 0.52 vs. 0.60–0.62 validation accuracy).
4. **Compare training with frozen layers and fine tuning.** Completed – see Task 4 (Section 15) and Section 16: partially unfreezing the base consistently and substantially outperformed keeping it fully frozen.
5. **Evaluate the model using Precision, Recall and F1-score.** Completed – see Task 5 (Section 15) and Section 18.1.
6. **Compare the performance of LeNet, AlexNet and ResNet.** Done in the Results Section while comparing all VGG with other models – comparison table in Section 18.2.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.

3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>